QwenQPU A Language Model with an Internal Quantum Circuit Simulator

OpenQuantum

Abstract

We present QwenQPU, a language model with a quantum-circuit simulator implemented within its standard layers. The model combines 32 language layers inherited from Qwen3.5-4B with four analytically constructed, fixed-weight layers. The language layers produce structured requests; the execution layers maintain addressable complex amplitudes in recurrent state, compose quantum gates through shared SwiGLU arithmetic, and correct probability readout under a specified norm condition. The added operators require no further training and run through the standard Transformers forward pass. We evaluate 48 frozen, paired tasks spanning 3, 4, and 6 logical qubits and 12, 18, and 24 sequential gates. Both conditions share language weights and a budget ceiling of 16,384 generated predictions, including every QPU forward prediction. Enabling execution increases the number of tasks satisfying an absolute error threshold of 5×10^{-5} for both requested event probabilities from 4/48 to 48/48. The maximum probability error is 1.42×10^{-5}, with a two-sided exact McNemar p-value of 1.14×10^{-13}. Under the fixed six-qubit, two-event protocol, QPU predictions follow $N_{\mathrm{QPU}} = 2{,}063 + 514G$, where G is the gate count; improved task success comes with more total predictions and longer runtime. These results demonstrate an executable and verifiable integration of explicit numerical rules with pretrained language capabilities, with accuracy and cost characterized within a finite-capacity, finite-precision setting.

Keywords: large language models; quantum circuit simulation; compiled weights; Gated DeltaNet; numerical execution; reproducible evaluation

1 Introduction

Language models used for scientific computation must translate natural-language requirements into verifiable numerical results. Toolformer learns to invoke external APIs [1], PAL and Program of Thoughts delegate computation to program interpreters [2], [3], and ReAct organizes sequences of reasoning and interaction with an environment [4]. These approaches highlight the importance of reliable execution. Here we ask whether explicit state-transition rules can be incorporated into a language model's standard forward computation to complete tasks that require continuously updated numerical state, subject to a specified error tolerance.

We introduce QwenQPU, which implements a state-vector simulator with a capacity of six qubits in four fixed-weight QPU layers and combines them with 32 inherited language layers in a single model. The language layers formulate circuit-execution requests; the QPU layers update complex amplitudes according to the gate sequence and return the probabilities of specified events. The design connects addressable state, gate arithmetic shared across addresses, and finite-precision readout into one execution path. The rules for H, X, T, and CNOT are specified by analytically constructed parameters. The added operators require no gradient-based training and run through the standard Transformers forward pass.

This design connects program representations in model weights with recurrent numerical state. RASP and Tracr provide a Transformer-oriented programming language and a mapping from programs to weights, respectively [5], [6]. Computational-completeness results characterize expressivity under specific assumptions [7], while Qwenputer demonstrates explicit instruction execution within standard Qwen layers [8]. For state maintenance, Neural Turing Machines and the Differentiable Neural Computer connect neural networks to differentiable memory [9], [10]. Linear attention, its fast-weight interpretation, and Gated DeltaNet link sequence computation

to recurrent storage and directed updates [11], [12], [13]. Numerical tasks impose further requirements: internal state must represent explicit variables, successive operations must preserve computational semantics, and final errors must be checked independently.

Quantum circuits impose all three requirements. Local gates compose into a global evolution [14], yet final probabilities depend on the phase, interference, and aggregation of complex amplitudes; explicitly representing an n-qubit register requires 2^n complex amplitudes. Classical simulation includes stabilizer methods that exploit Clifford structure [15], restricted-entanglement and tensor-network methods that exploit state or circuit structure [16], [17], and dedicated simulators such as QuEST and Qulacs [18], [19]. Neural quantum states instead investigate learnable wavefunction representations [20]. QwenQPU uses a full state vector and maps explicit quantum-evolution rules into a language model's computational structure, giving each storage, arithmetic, and output operation a correspondence to an inspectable mathematical quantity.

The method has three interdependent components. An addressable two-digit encoding and double-buffered state store complex amplitudes as numerical variables that can be read and written individually. Arithmetic shared across addresses reduces gate operations to address permutations and real-valued sums and differences, avoiding a separate copy of the arithmetic for each amplitude. Model-internal event aggregation and a near-unit-norm correction connect state evolution to probability outputs under explicit error conditions. We also specify the roles of the four layers, the host-interface boundary, and the number of forward predictions, so that both implementation and execution cost can be examined.

We test whether this execution path translates into task capability on a frozen benchmark. The 48 paired tasks span 3, 4, and 6 logical qubits and 12, 18, and 24 sequential gates, including Clifford and non-Clifford circuits. With language weights fixed and the same ceiling of 16,384 predictions, enabling execution increases the number of successful tasks from 4/48 to 48/48, with a maximum probability error of 1.42×10^{-5}. This experiment establishes a system-level capability gain from the execution mechanism and its interface. Numerical checks and complete resource records characterize the accuracy, capacity, and cost conditions under which that gain holds.

2 Model overview

2.1 Task definition and numerical semantics

Consider an n-qubit register initialized in the all-zero state and a sequential circuit composed of H, X, T, and CNOT gates. Let G denote the gate count. The final state and the probability of a measurement event are defined as

$$|\psi_G\rangle = U_G \cdots U_2 U_1 |0\rangle^{\otimes n}, \qquad p(e) = \sum_{x\in\{0,1\}^n} \mathbf{1}[x \models e]\, |\langle x|\psi_G\rangle|^2. \tag{1}$$

An event e is a conjunction of conditions specifying qubit indices and measurement values; unspecified qubits are marginalized by summation. Index 0 denotes the least significant bit in the integer encoding of a basis state. Each task requests the probabilities of two specified events, A and B. The input supplies the circuit and gate order, so the evaluation concerns interpretation and execution of a given circuit rather than circuit synthesis from an abstract physical objective.

The evaluated implementation uses a single six-qubit storage capacity. Three- and four-qubit tasks are embedded in this register, with the remaining qubits kept in their initial state. Main-experiment probabilities are obtained by numerical readout of the stored state vector, without shot-based estimation. Correctness is defined by an explicit absolute-error threshold. The full-

state representation retains the complex amplitudes needed for interference, while independent reference values assess the finite precision introduced by FP32 arithmetic and digit encoding.

2.2 Model composition

QwenQPU retains 32 language layers inherited from Qwen3.5-4B [21] through the pinned upstream checkpoint and replaces its 11-layer CPU prefix with four QPU layers [22]. The complete model uses the standard Qwen3_5ForCausalLM class and contains 36 layers and 9,167,926,528 stored parameters, corresponding to an FP32 tensor payload of 36,671,706,112 bytes. Assembly uses a common 4096-dimensional residual stream: the first 2560 coordinates carry the language component and the next 1024 carry the quantum component, with corresponding adjustments to projections and normalization scales [22]. The residual width, attention-head layout, feedforward shapes, and expanded vocabulary determine the storage footprint; zero padding in dense tensors is included in the parameter count. Thus, 9.17B denotes the stored parameters of the assembled model and does not represent approximately 5B newly trained parameters added to a 4B language model. Appendix A identifies the upstream version and assembly components.

Table 1. QwenQPU model specifications. Stored parameters include inherited language weights and analytically constructed QPU weights. Numerical settings are those used in the evaluation.

Property	Specification
Language-layer source	Qwen3.5-4B, inherited through a fixed Qwenputer checkpoint
Composition	32 inherited language layers + 4 fixed-weight QPU layers
Standard model class	Qwen3_5ForCausalLM
Stored parameters, including zero padding	9,167,926,528
Simulated register capacity	6 qubits; 64 complex amplitudes
Supported gates	H, X, T, CNOT
State and arithmetic	Gated DeltaNet recurrent state; fixed SwiGLU projections
Numerical settings	FP32; TF32 disabled
Probability output	Marginal probability of a specified computational-basis measurement event

2.3 Two-stage inference linking language and execution

QwenQPU uses two connected inference stages. The language stage organizes the supplied circuit and measurement conditions into a structured request. The execution stage updates the simulated quantum state according to the loaded gate sequence and reads out event probabilities. Both stages use the same assembled checkpoint, and every forward call executes the complete standard model; the relevant residual coordinates and control protocol determine the active function. The host parses requests, loads instructions, and schedules calls. The model's numerical output is returned directly as the final answer, without a second language-generation stage.

3 Model construction and execution

QwenQPU has two parameter sources: language layers inherited from a pretrained checkpoint and QPU layers analytically constructed from gate coefficients, address logic, digit encoding,

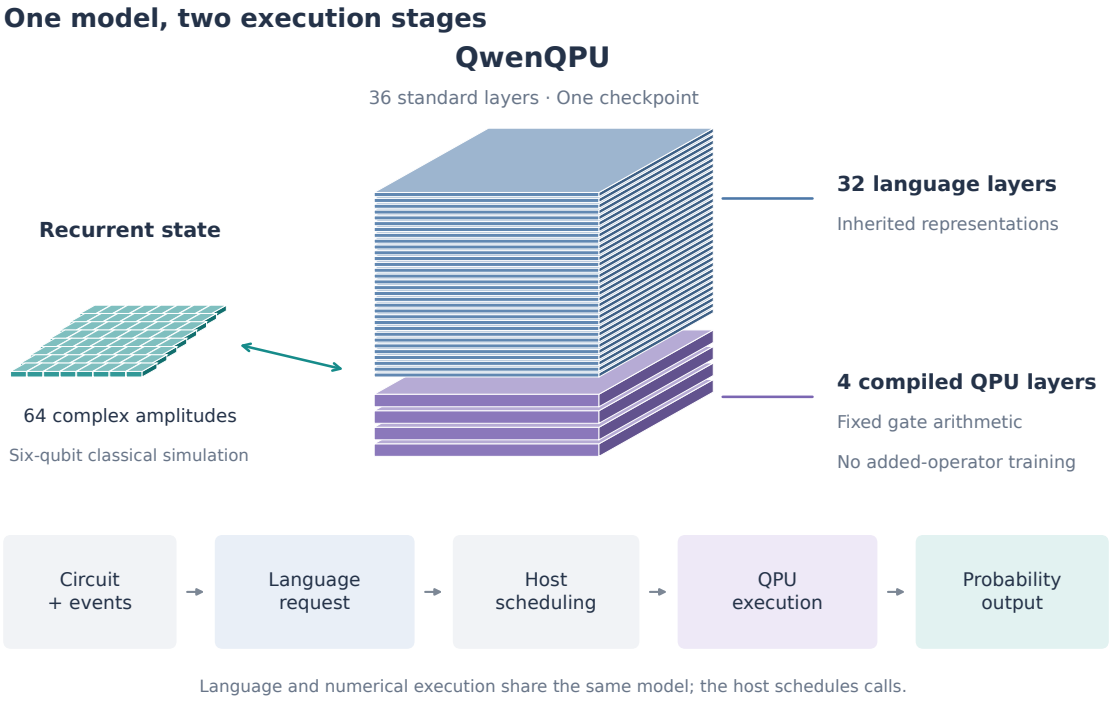


Figure 1. Language interface and quantum circuit execution within one model. The three-dimensional stack represents 32 inherited language layers and four fixed-weight QPU layers; in the actual forward order, the QPU layers precede the language layers. Recurrent state stores the 64 complex amplitudes associated with a six-qubit capacity. The lower panel connects circuit and event inputs, language requests, host scheduling, model-internal execution, and probability outputs. The host handles parsing, instruction loading, scheduling, and digit decoding; standard model layers perform amplitude updates and probability arithmetic. The three-dimensional structure is an architectural schematic of state-vector simulation on classical hardware.

and readout rules. The construction procedure maps these rules to projections and recurrent updates in standard layers and assembles them with the language layers; the added execution capability uses no optimizer or additional training data [22]. Across repeated forward calls, the four QPU layers jointly implement storage, computation, and output, with roles selected by the gate-execution or probability-query phase (Table 2). We describe state encoding, shared arithmetic, and readout error conditions in turn.

These roles follow the fixed-version four-layer construction code and readout-correction implementation. Gate execution and probability queries reuse these layers across multiple forward calls and digit-feedback steps; a depth of four layers does not imply that four calls execute an entire circuit.

3.1 Addressable amplitudes in recurrent state

Work on linear attention and fast weights establishes the role of recurrent matrices as key–value memory [11], [12]. We use Gated DeltaNet to combine state retention and directed writes in recurrent updates [13]. An idealized single-head matrix update illustrates this storage role. With state matrix S, key k, and value v, the update can be written as

$$\widetilde{S}_t = \alpha_t S_{t-1}, \qquad S_t = \widetilde{S}_t + \beta_t k_t (v_t^{\mathsf{T}} - k_t^{\mathsf{T}} \widetilde{S}_t). \tag{2}$$

With retention and overwrite settings for the state gate and write strength, one-hot keys address updates to individual storage rows. The compiler uses this mechanism to organize am-

Table 2. Main roles of the four QPU layers in gate execution and probability readout. Layers are numbered from 1. The protocol phase selects which arithmetic units are active within a layer. The table describes the probability-execution path used in this study.

Layer	State or arithmetic role	Quantities passed to subsequent steps
QPU layer 1	Gated DeltaNet maintains amplitude RAM, instruction ROM, control state, and accumulators. The MLP decodes reads and performs shared gate arithmetic; correction queries decode event mass E and total mass T.	Candidate updated amplitude digits, control values, real and imaginary components, accumulator values
QPU layer 2	Converts amplitude digits and control values to Gray codes, forms Born terms from real and imaginary components, and checks control conditions. Correction queries compute E(2−T).	Digit and control codes, Born terms, corrected event probability
QPU layer 3	Combines Born terms with the accumulator to form probability-digit candidates, or forms candidates from the corrected probability; applies control and query validity constraints.	High- and low-order probability-digit candidates and output constraints
QPU layer 4	Converts probability-digit candidates to Gray codes and works with the standard output head to produce digit tokens.	Decodable probability digits, fed back or returned according to the protocol

plitude memory, instruction ROM, the program counter, and accumulators. Replicated storage ports provide the multiple reads required by arithmetic. A quantum state contains 2^{n} complex amplitudes, and each real or imaginary component is represented by two base-4096 digits:

$$r = \frac{4096 d_{\mathrm{hi}} + d_{\mathrm{lo}}}{2^{22}} - 1, \qquad d_{\mathrm{hi}}, d_{\mathrm{lo}} \in \{0, \ldots, 4095\}. \tag{3}$$

The resulting numerical grid spacing is 2^{-22}. Digit tokens carry the high- and low-order digits, and the digits emitted by gate operations are fed back into recurrent state to complete the next storage update. Double buffering separates a gate's output from its input state. Arithmetic is reused across addresses, so a separate set of gate-computation rules is not required for each amplitude address.

3.2 Gate operations through shared arithmetic

The gate compiler maps H, X, T, and CNOT to address selection and a small set of shared arithmetic forms. For a pair of amplitudes whose target bit is 0 or 1, H computes the normalized sum and difference. T multiplies an amplitude with target bit 1 by $\exp(\mathrm{i}\pi/4)$, which reduces to sums and differences of its real and imaginary components:

$$H: \quad (a_0, a_1) \mapsto \left(\frac{a_0 + a_1}{\sqrt{2}}, \frac{a_0 - a_1}{\sqrt{2}} \right), \qquad T: \quad a + ib \mapsto \frac{a - b}{\sqrt{2}} + i\frac{a + b}{\sqrt{2}}. \tag{4}$$

X selects source amplitudes by flipping the target address bit; CNOT applies this permutation conditionally on the control bit. The SwiGLU feedforward structure includes an elementwise product of an activated branch and a linear branch [23]. We set its projections to analytically specified coefficients and reuse the same arithmetic across gates, addresses, and digit phases; initialization uses the corresponding constant form. The compiler also constructs control tokens, program counting, and stopping conditions, allowing gate sequences to advance through model outputs and feedback [22].

H, T, and CNOT provide universal approximation in the ideal gate-sequence model [24]. The scope of the present implementation and experiments is jointly determined by six-qubit storage, the evaluated gate counts, and finite-precision conditions.

3.3 Born readout and normalization correction

For each basis state, the execution layers compute the sum of the squared real and imaginary components and accumulate it within the model according to an event mask. Let E denote event mass and T total mass. Because finite precision can cause T to deviate from 1, readout uses the following correction near unit norm:

$$\widehat{p} = E(2 - T), \qquad |T - 1| \leq 5 \times 10^{-4}. \tag{5}$$

The multiplication is implemented through SwiGLU's multiplicative structure; the identity SiLU(z)−SiLU(−z)=z provides a basis for constructing the required linear factor. For T>0 and 0≤E≤T, the difference between the corrected value and the normalized ratio satisfies

$$\left| \frac{E}{T} - E(2 - T) \right| = \frac{E}{T}(T - 1)^2 \leq 2.5 \times 10^{-7}. \tag{6}$$

This bound applies only to the normalization approximation. The complete output also includes errors from amplitude evolution, digit quantization, and floating-point arithmetic, so final probabilities are evaluated against an independent state-vector calculation. The host checks the stated norm range and transports digits; event arithmetic and the correction are performed in the fixed layers.

3.4 Execution protocol and host interface

We define model-internal execution operationally: model parameters and standard forward operators implement amplitude updates, Born probabilities, and event summation. The host parses gate strings and event indices, loads instructions, builds query masks, schedules calls, selects output tokens, and decodes the final digits. It does not use an external state-vector simulator to generate execution answers. Qiskit and NumPy provide independent validation. The benchmark uses native function-call syntax: the function name provides a scheduling entry point backed by the QPU layers of the same model [25].

4 Evaluation

4.1 Frozen tasks and independent references

The primary benchmark contains 36 non-Clifford tasks: four circuits for each of the nine combinations of 3, 4, or 6 logical qubits and 12, 18, or 24 sequential gates. An additional 12 Clifford control tasks contain four 18-gate circuits at each register size. Every task starts from the all-zero state. Event A specifies one bit and event B specifies two bits, with the remaining bits marginalized [25].

Before model inference, candidate circuits are screened using physical criteria: a connected interaction graph, entanglement in the final state, and event probabilities in [0.025,0.975]. Non-Clifford tasks additionally require at least one event probability to be more than 0.001 from every multiple of 1/16. Screening does not use model answers. Tasks and the evaluation protocol are frozen before inference. Independent NumPy and Qiskit Statevector implementations [26] agree across all tasks and two additional interface probes, with a maximum probability difference of 6.66×10^{-16} [25]. The probes are excluded from the 48 main tasks.

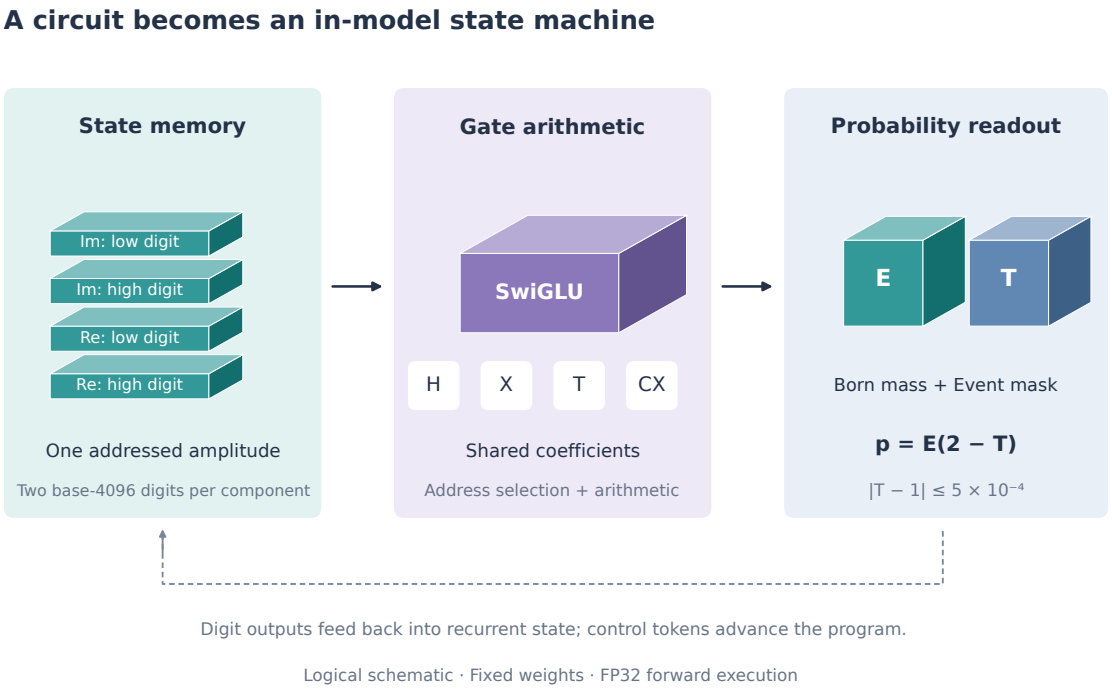


Figure 2. Model-internal execution from amplitude digits to measurement probabilities. The real and imaginary parts of each complex amplitude are each encoded by two base-4096 digits; the slabs represent digit components. Fixed SwiGLU projections and address selection implement H, X, T, and CNOT (CX), with output digits fed back to recurrent state. Readout accumulates event mass E and total mass T and applies $\widehat{p}$=E(2−T) when |T−1|≤5×10^{-4}. The schematic groups functions into storage, arithmetic, and readout; these blocks do not correspond one-to-one to separate physical devices or model layers. The added operator weights are analytically constructed.

4.2 Paired conditions and budgets

Both conditions use the same assembled checkpoint and tasks. The baseline disables the quantum-execution entry point and obtains probabilities directly from the language component. QwenQPU receives a native function definition and returns numerical answers from its own QPU after submitting the circuit and events. The shared system prompt requires immediate function use when available and self-contained calculation otherwise. This paired design holds language weights fixed while changing the availability of execution and its interface. The term "Qwen baseline" denotes this disabled condition throughout. The experiment does not include a condition in which the same language model calls an external simulator.

Each condition has a total prediction budget of 16,384. Language tokens, function arguments, and every native QPU forward prediction count toward this ceiling, which is not reset after a function call; host-supplied control tokens are recorded separately. The two conditions share a generation seed for each task. Task order is shuffled with a fixed seed, and the condition executed first is alternated.

Language decoding uses the native non-thinking template with temperature=1, top-p=0.95, top-k=20, presence penalty=1.5, and repetition penalty=1. The time limit is 3,600 seconds per condition, and completed failures are not regenerated. The experiment is a single-seed paired evaluation on a fixed task set.

4.3 Success criteria

Numerical success requires both event probabilities, A and B, to differ from the Qiskit complex128 reference by no more than 5×10^{-5}. QwenQPU must also submit the same event sets as the task, and the submitted circuit must produce a final-state infidelity below 10^{-10} for the specified initial state. This circuit criterion verifies agreement for that initial state; it does not require operator equivalence on all possible inputs.

Formatting errors, contradictory answers, missing probabilities, truncation, and timeouts remain in the denominator. Identical repeated JSON answers may be consolidated; conflicting answers count as failures. The statistical unit is the task, and the two events within a task are not treated as independent samples. We report Wilson intervals for success rates [27], two-sided exact conditional McNemar tests [28], [29], and conservative intervals for paired differences based on Clopper–Pearson intervals [30]. Appendix B specifies the calculations.

4.4 Implementation and measurement

Evaluation uses an NVIDIA A100 80 GB GPU, PyTorch 2.6.0+cu124, and Transformers 5.19.0, with numerical execution in FP32 and TF32 disabled. Timing includes language decoding, native execution, and progress input/output, is measured after CUDA synchronization, and excludes model loading. Peak allocated GPU memory is 38.56 GB for the baseline and 38.40 GB for QwenQPU. "Equal budget" refers to the same prediction-count ceiling, not to equal realized prediction counts, FLOPs, or runtime.

5 Model capabilities and execution cost

5.1 Internal execution satisfies all criteria on 48 frozen tasks

QwenQPU meets the numerical and request-fidelity criteria on all 48 frozen tasks, compared with four for the baseline, a difference of 91.67 percentage points. All 44 discordant pairs favor QwenQPU; both conditions are correct on the remaining four tasks. The non-Clifford primary endpoint improves from 2/36 to 36/36 (Table 3), showing that the gain extends to the evaluated circuits containing T gates.

Table 3. Paired results on the frozen task set. Success requires both event probabilities to pass. Non-Clifford and Clifford rows form disjoint subsets of the benchmark.

Task family	Tasks	Qwen baseline	QwenQPU
Non-Clifford primary endpoint	36	2/36, 5.6%	36/36, 100%
Clifford controls	12	2/12, 16.7%	12/12, 100%
All tasks	48	4/48, 8.3%	48/48, 100%

The conservative exact 95% interval for the overall paired improvement is 69.50–98.12 percentage points, with a two-sided exact McNemar $p=1.1369\times10^{-13}$. The improvement on the non-Clifford primary endpoint is 94.44 percentage points, with $p=1.1642\times10^{-10}$. Appendix B provides success-rate intervals and further statistical details. These statistics describe the single-seed paired results for the frozen task family; inference across task distributions or seeds requires corresponding replication.

By logical register size, the baseline succeeds on 3/16, 0/16, and 1/16 tasks at 3, 4, and 6 qubits, respectively, whereas QwenQPU succeeds on 16/16 at each size. In the non-Clifford subsets with 12, 18, and 24 gates, the baseline succeeds on 1/12, 1/12, and 0/12, respectively,

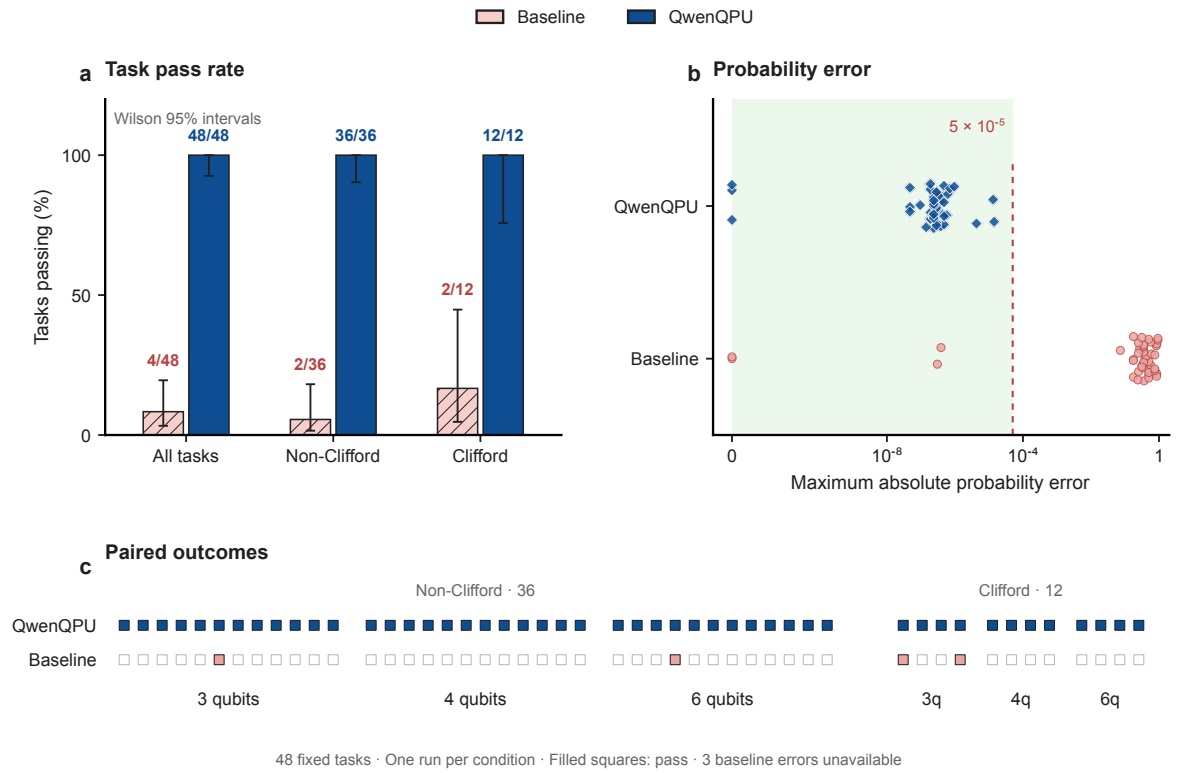


Figure 3. Paired results for 48 frozen tasks. a, Grouped bars show success rates from a zero baseline, labeled with exact successful-task counts and denominators; black intervals are Wilson 95% intervals. "All" combines 36 non-Clifford and 12 Clifford tasks. b, Each point shows the larger absolute error of the two event probabilities in one run: 48 QwenQPU points and 45 valid numerical baseline outputs. The other three baseline failures have no valid numerical error but remain in the success-rate denominator. The symmetric-log horizontal axis retains a linear region near zero (threshold 10^{-12}); the red dashed line marks the 5×10^{-5} criterion, and the pale-green region lies within it. c, Each column pairs the two conditions for one task; filled squares indicate success and open squares failure. Conditions share language weights and a ceiling of 16,384 predictions, with one run per task per condition. All 96 records are retained: 44 failures become successes and four existing successes are preserved. Vertical jitter in the error scatter reduces overlap without changing error values. Data comprise all r5 records from the fixed version [25].

versus 12/12 for QwenQPU in each subset. These strata describe different logical task sizes within the same six-qubit storage model; they do not test scalability under an increase in model storage capacity.

5.2 Probability errors and representative circuits meet numerical requirements

Across the 48 tasks, QwenQPU's maximum absolute probability error is 1.4209657×10^{-5}, below the prespecified threshold of 5×10^{-5}, and every request reaches numerical completion within the prediction budget. The baseline's 44 failures comprise 41 numerical errors, two conflicting answers, and one answer missing the required A/B probabilities; all 48 baseline generations terminate normally with EOS. Baseline failures therefore primarily manifest as probability outputs that do not meet the numerical tolerance. This output classification alone does not identify the internal cause of the errors.

Two complete-model demonstrations test phase interference and joint readout across multiple qubits. H3–T3–H3 converts a phase difference into measurement probabilities, while a six-qubit GHZ circuit tests a CNOT chain and joint-event aggregation. In both demonstrations, the

language model first generates a request and the QPU layers of the complete assembled model return the numerical answer. These demonstrations are excluded from the 48 tasks in Table 3 [22].

Table 4. Independent demonstrations with the complete model. Qubit indices start at 0, and both circuits begin in the six-qubit all-zero state.

Circuit and event	QwenQPU	Reference probability	Absolute error
H3–T3–H3, q3=1	0.1464464664	0.1464466094	1.43×10^{-7}
H3–T3–H3, all zeros	0.8535535336	0.8535533906	1.43×10^{-7}
H0 followed by CNOT chain 0→1→2→3→4→5, all zeros	0.5	0.5	1.11×10^{-16}
Same GHZ circuit, all ones	0.5	0.5	1.11×10^{-16}

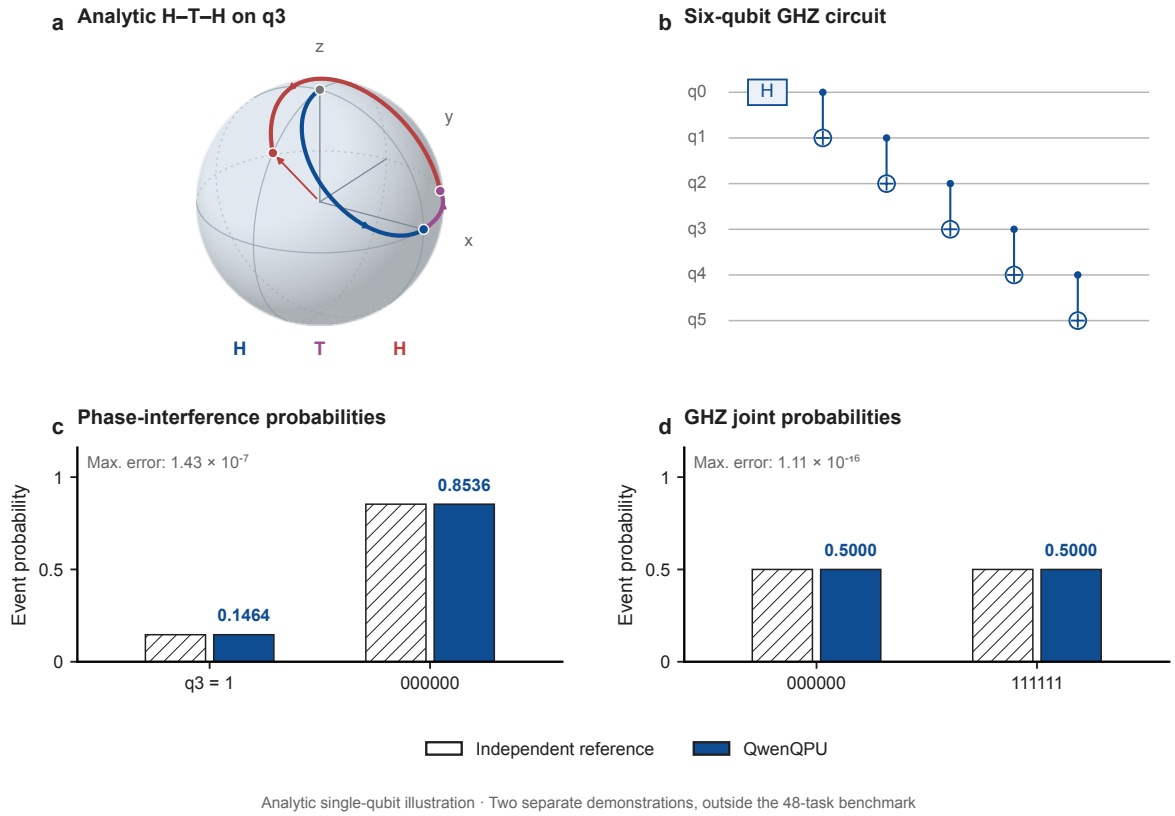


Figure 4. Phase interference and six-qubit joint probabilities. a, Analytical Bloch-sphere schematic of H–T–H on active qubit q3, initially in $|0\rangle$. Surface shading represents geometric illumination and arcs represent analytical rotations, not measured internal model trajectories. b, H0 and successive CNOT gates form a six-qubit GHZ circuit. c,d, Hatched open bars show independent reference probabilities and dark-blue bars show saved complete-model outputs, all on a 0–1 probability scale. Blue labels report rounded model outputs. Maximum absolute errors are 1.43×10^{-7} and 1.11×10^{-16}, respectively. Each circuit starts in the six-qubit all-zero state and has one complete-model demonstration; neither is included in the 48 tasks in Figure 3, and no repeated-measurement error bars are shown. The GHZ panel tests computational-basis joint probabilities of the ideal simulated circuit.

These demonstrations check output agreement for phase-gate and controlled-gate composition and event aggregation in two interpretable circuits. The GHZ result concerns only

computational-basis joint probabilities; independent certification of coherence or entanglement requires additional observables.

5.3 Normalization correction reduces readout error on a saved final state

To examine how finite-precision norm drift affects probability readout, we compare uncorrected and corrected outputs from the same saved final state. For the state saved after a six-qubit, 1,000-gate run, the uncorrected probability error for the event containing all outcomes is approximately 1.21×10^{-4}. After applying the correction in Section 3.3, the maximum error across nine reference comparisons is 1.072×10^{-5}. This result supports the readout correction for that saved state near unit norm. Because the full 1,000-gate circuit was not rerun with all corrections incorporated, this is not an end-to-end validation of long-circuit accuracy. Task-success results in Table 3 come from independent complete runs of the 12-, 18-, and 24-gate circuits.

5.4 Execution cost is explicitly determined by gate count

Under the current protocol with six-qubit capacity and two measurement events, the number of QPU predictions is a deterministic function of the sequential gate count:

$$N_{\mathrm{QPU}}(G) = 2063 + 514G. \tag{7}$$

Tasks with 12, 18, and 24 gates require 8,231, 11,315, and 14,399 QPU predictions, respectively, totaling 543,120 across the 48 tasks. Every task agrees with this expression. The relation measures cost under a fixed execution protocol; a change in qubit capacity, the number of readout events, or scheduling rules requires the corresponding cost to be recalculated.

Table 5. Actual resource use for 48 paired tasks. Each condition includes all 48 runs, including successes and failures. Amortized time per correct task is total runtime divided by the number of correct tasks, not the mean latency of successful runs.

Metric	**Qwen baseline**	**QwenQPU**
Total language predictions	238,639	8,007
Total QPU predictions	0	543,120
Combined predictions	238,639	551,127
Total end-to-end runtime, s	11,138.90	24,974.09
Median task runtime, s	1.87	524.20
Amortized time per correct task, s	2,784.72	520.29

The accuracy gain comes with greater actual execution expenditure: QwenQPU uses 551,127 predictions, approximately 2.31 times the baseline's 238,639. Total runtime rises from 11,138.90 s to 24,974.09 s, and median task runtime rises from 1.87 s to 524.20 s (Table 5 and Figure 5). Language predictions decrease to 8,007, while numerical computation is carried by 543,120 QPU predictions. Amortized time, calculated by dividing total runtime by the number of correct tasks, decreases from 2,784.72 s to 520.29 s. This quantity reflects the rate of producing correct results on the current task set; it does not establish faster individual responses or greater efficiency than an external simulator.

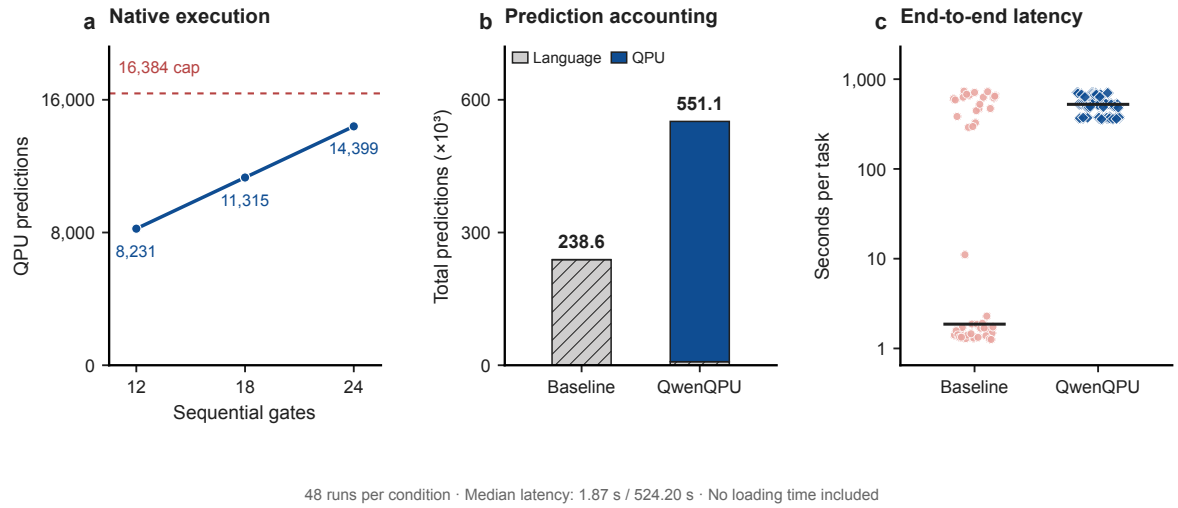


Figure 5. Execution cost and measured runtime. a, The fixed six-qubit, two-event protocol satisfies $N_{\mathrm{QPU}} = 2{,}063 + 514G$; the three points correspond to 12, 18, and 24 gates. The red dashed line marks the shared total ceiling of 16,384 predictions, including language and QPU predictions. b, Total language and native QPU predictions across all 48 tasks, retaining successful and failed runs: 551,127 for QwenQPU and 238,639 for the baseline. Bar values are in thousands. c, Each point is the end-to-end runtime of one task, with 48 points per condition. The vertical axis is logarithmic, and black horizontal lines show medians of 1.87 s and 524.20 s. Timing excludes model loading and includes language generation, native execution, and progress input/output. Under these conditions, the capability gain comes with more total predictions and longer total runtime.

6 Discussion

QwenQPU shows that analytical numerical rules can form an executable model together with a pretrained language interface when connected through explicit state representations and a readout protocol. Addressable recurrent state associates each amplitude component with an inspectable storage location. Shared arithmetic lets the same projections implement gate operations at different addresses, and the readout correction connects finite-precision state to final probabilities. Together, these components establish a correspondence between internal mathematical variables and task outputs that can be tested through independent references and complete execution records.

Building on program-to-weight construction and recurrent-memory research [5], [6], [13], this work extends validation to successive complex-state evolution and event-probability aggregation. A central design lesson is that execution rules, numerical encoding, and error conditions must be developed together. Fixed gate rules do not automatically guarantee numerical accuracy for arbitrarily long circuits. The saved-state correction result further motivates separate evaluation of state-evolution errors and probability-aggregation errors.

The paired experiment establishes the contribution of access to numerical execution on the evaluated tasks, but does not distinguish the benefit of having a reliable executor from the benefit of placing that executor inside the model. Assessing execution location itself requires a matched condition in which the same language model calls an external simulator through the same request protocol. Tasks, scoring, and resource accounting should be held consistent before comparing accuracy, latency, and deployment overhead. This is a direct next comparison for evaluating the value of model-internal execution.

The evidence covers six-qubit simulation capacity, the H/X/T/CNOT gate set, the all-zero initial state, ideal terminal measurement, and a fixed task family. Three- and four-qubit tasks use the same six-qubit storage, so the results do not establish cost scaling under increased capacity;

explicit state-vector storage still grows as 2^n. Each condition has one generation, leaving broader task distributions, multiple initial states, and stability across seeds to be evaluated. QPU here denotes a state-vector execution component on classical hardware; the study provides no evidence of physical quantum speedup.

Within these bounds, QwenQPU provides a concrete system for studying numerical execution inside language models. Capabilities can be traced to known rules, errors can be separated into state and readout contributions, and execution expenditure can be counted under a specified protocol. Capacity extensions and implementation improvements can use these verifiable quantities to examine the relationship between accuracy, reliability, and resource cost.

7 Conclusion

QwenQPU implements addressable complex amplitudes, shared quantum-gate arithmetic, and probability readout in four fixed-weight layers, combining them with pretrained language layers for circuit interpretation and numerical execution. On 48 frozen paired tasks, enabling execution increases the number of successful tasks from four to 48, with a maximum probability error of 1.42×10^{-5}. Prediction counts increase linearly with gate count under the fixed protocol, and the model consumes more total predictions and runtime than the baseline. Within six-qubit capacity and the evaluated circuit range, this work establishes an implementation path from analytical numerical rules through standard model forward computation to verifiable task results, providing a reproducible basis for studying the accuracy and cost of model-internal execution.

Code and data availability

The compiler, model-assembly code, task interface, frozen task list, per-condition result extracts, and scoring code are available in the QwenQPU code repository. The main experiment uses the standard added-layer implementation in research/standard_layers, with results in evidence/added-layers. Main tables can be reconstructed from the 96 released condition records; rerunning model generation requires the corresponding checkpoints and runtime environment. The experiment is identified jointly by a fixed code commit, the upstream language-checkpoint revision, and hashes of the four-layer QPU prefix, as specified in Appendix A.

The model distribution entry point is xi-zhao/QwenQPU. The fixed-version experiment manifest records a model assembled and executed in memory. Reproduction should verify its components and weight hashes rather than substitute the distribution repository's mutable main branch for the experimental model identity. Establishing that a complete serialized checkpoint matches this identity requires a corresponding release manifest and readback-validation records. Sources of inherited weights, construction methods, and code components are documented in the attribution and licensing statement.

References

[1] Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems, 36, 2023. Publication record. arXiv:2302.04761.

[2] Gao L, Madaan A, Zhou S, et al. PAL: Program-aided Language Models. Proceedings of ICML, PMLR 202:10764–10799, 2023. Article.

[3] Chen W, Ma X, Wang X, Cohen W W. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. Transactions on Machine Learning Research, 2023. Article.

[4] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. Article.

[5] Weiss G, Goldberg Y, Yahav E. Thinking Like Transformers. Proceedings of ICML, PMLR 139:11080–11090, 2021. Paper and publication record.

[6] Lindner D, Kramár J, Farquhar S, Rahtz M, McGrath T, Mikulik V. Tracr: Compiled Transformers as a Laboratory for Interpretability. Advances in Neural Information Processing Systems, 36, 2023. Publication record. arXiv:2301.05062.

[7] Pérez J, Barceló P, Marinkovic J. Attention is Turing-Complete. Journal of Machine Learning Research, 22(75):1–35, 2021. Article.

[8] Agrawal D. Qwenputer: a fully functioning, deterministic RISC-V CPU inside Qwen3.5. 22 October 2026. Technical blog post. Accessed 11 November 2026. Project report.

[9] Graves A, Wayne G, Danihelka I. Neural Turing Machines. arXiv:1410.5401, 2014. Article.

[10] Graves A, Wayne G, Reynolds M, et al. Hybrid computing using a neural network with dynamic external memory. Nature, 538(7626):471–476, 2016. DOI: 10.1038/nature20101. Article.

[11] Katharopoulos A, Vyas A, Pappas N, Fleuret F. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. Proceedings of ICML, PMLR 119:5156–5165, 2020. Article.

[12] Schlag I, Irie K, Schmidhuber J. Linear Transformers Are Secretly Fast Weight Programmers. Proceedings of ICML, PMLR 139:9355–9366, 2021. Article.

[13] Yang S, Kautz J, Hatamizadeh A. Gated Delta Networks: Improving Mamba2 with Delta Rule. ICLR, 2025. arXiv:2412.06464.

[14] Barenco A, Bennett C H, Cleve R, et al. Elementary gates for quantum computation. Physical Review A, 52:3457–3467, 1995. DOI: 10.1103/PhysRevA.52.3457. Article.

[15] Aaronson S, Gottesman D. Improved Simulation of Stabilizer Circuits. Physical Review A, 70:052328, 2004. DOI: 10.1103/PhysRevA.70.052328. Article.

[16] Vidal G. Efficient classical simulation of slightly entangled quantum computations. Physical Review Letters, 91:147902, 2003. DOI: 10.1103/PhysRevLett.91.147902. Article.

[17] Markov I L, Shi Y. Simulating quantum computation by contracting tensor networks. SIAM Journal on Computing, 38(3):963–981, 2008. DOI: 10.1137/050644756. Article.

[18] Jones T, Brown A, Bush I, Benjamin S C. QuEST and High Performance Simulation of Quantum Computers. Scientific Reports, 9:10736, 2019. DOI: 10.1038/s41598-019-47174-9. Article.

[19] Suzuki Y, Kawase Y, Masumura Y, et al. Qulacs: a fast and versatile quantum circuit simulator for research purpose. Quantum, 5:559, 2021. DOI: 10.22331/q-2021-10-06-559. Article.

[20] Carleo G, Troyer M. Solving the Quantum Many-Body Problem with Artificial Neural Networks. Science, 355:602–606, 2017. DOI: 10.1126/science.aag2302. Article.

[21] Qwen Team. Qwen3.5-4B model card. 2026. Accessed 11 November 2026. Official model card.

[22] QwenQPU contributors. QPU in standard Qwen layers. Project documentation, commit `caae45327b46458163d0840a4c1466e5575a7b4f`, 2026. Implementation and demonstrations.

[23] Shazeer N. GLU Variants Improve Transformer. arXiv:2002.05202, 2020. Article.

[24] Boykin P O, Mor T, Pulver M, Roychowdhury V, Vatan F. On Universal and Fault-Tolerant Quantum Computing. arXiv:quant-ph/9906054, 1999. Article.

[25] QwenQPU contributors. Supplied-circuit probability benchmark (r5). Project documentation and data, commit `caae45327b46458163d0840a4c1466e5575a7b4f`, 2026. Tasks and evaluation protocol; Per-condition results; Summary statistics.

[26] IBM Quantum. Qiskit Statevector API documentation. Accessed 11 November 2026. Official documentation.

[27] Wilson E B. Probable Inference, the Law of Succession, and Statistical Inference. Journal of the American Statistical Association, 22(158):209–212, 1927. DOI: 10.1080/01621459.1927.10502953. Article.

[28] McNemar Q. Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2):153–157, 1947. DOI: 10.1007/BF02295996. Article.

[29] Fagerland M W, Lydersen S, Laake P. The McNemar test for binary matched-pairs data: mid-p and asymptotic are better than exact conditional. BMC Medical Research Methodology, 13:91, 2013. DOI: 10.1186/1471-2288-13-91. Article.

[30] Clopper C J, Pearson E S. The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika, 26(4):404–413, 1934. DOI: 10.1093/biomet/26.4.404. Article.

A Reproduction settings and versions

Table A1. Reproduction settings and fixed versions.

Item	Setting
Language source	Qwen3.5-4B, inherited through a fixed Qwenputer checkpoint
Upstream checkpoint	divagr1925/Qwen3.5-4B-Qwenputer
Code commit	`caae45327b46458163d0840a4c1466e5575a7b4f`
Upstream revision	`2d9068508f2b680badb7abcded96fdd107e9ba5d`
Standard-layer model	32 language layers and 4 QPU layers
Numerical mode	FP32; TF32 disabled
Software	PyTorch 2.6.0+cu124; Transformers 5.19.0
Physical compute device	NVIDIA A100 80 GB GPU
Simulated register capacity	6 qubits
Evaluation size	48 tasks; 96 paired-condition records
Numerical criterion	Absolute error $\leq 5\times10^{-5}$ for both events
Request criterion	Matching events; final-state infidelity $<10^{-10}$ from the all-zero initial state
Budget and time limit	16,384 generated predictions; 3,600 s per condition

The four-layer QPU prefix consists of two FP32 safetensors files. The hashes below come from the weight manifest at the fixed code version. They distinguish the execution prefix from the language source and the complete assembled model; the two prefix files alone do not contain the full language model.

Table A2. QPU prefix files and SHA-256 identities.

QPU prefix file	Bytes	SHA-256
`model-00001-of-00002.safetensors`	985,465,312	`d92ff7c27df6b4f0c3eb97821652a77fb38be34b72b272ee0d8e53bf363f6178`
`model-00002-of-00002.safetensors`	437,023,752	`03a3c8ce85ba089a4ea17aa88b8324e911d67a2c0801cc83e530071493d27730`

The frozen task list has SHA-256 `b6d892f5441f7b66fcc63d3c4a194188b5b84c410376225ae9d779feb01d0759`; the per-condition extract has SHA-256 `051643542579d3c78719c4db2fd6a3d777d5d447e6b8acccbf6ae2e94c52b9f6`. Task counts, success counts, errors, token counts, and runtimes should be checked against the same frozen version.

The following commands construct the prefix and run the complete-model demonstration provided in the source code. Output directories must be new, and the upstream checkpoint must already be available locally. Rerunning inference for the main benchmark also requires its frozen run configuration; the demonstration commands do not rerun all 48 tasks.

```
python research/standard_layers/build_prefix.py --out ./models/qpu-prefix
python research/standard_layers/demo_repaired_full_model.py \
  --base ./models/Qwen3.5-4B-Qwenputer \
  --prefix ./models/qpu-prefix \
  --out ./reproduced/added-layer-demo
```

B Paired statistics and result reconstruction

Let b count pairs for which QwenQPU is correct and the baseline is incorrect, and c count pairs with the reverse outcome. Under the null hypothesis, McNemar's exact conditional test models the directional count, conditional on b+c discordant pairs, as binomial with success probability 1/2 [28], [29]. Overall, b=44 and c=0, yielding a two-sided exact McNemar probability of $2\times2^{-44}=1.1368683772161603\times10^{-13}$. For the non-Clifford primary endpoint, b=34 and c=0, yielding $2\times2^{-34}=1.1641532182693481\times10^{-10}$. The paired improvements are 44/48 and 34/36, respectively.

Wilson 95% intervals for overall success rates are 3.29%–19.55% for the baseline and 92.59%–100% for QwenQPU. Paired-difference intervals use a Bonferroni joint construction from two 97.5% Clopper–Pearson marginal intervals for the probabilities of the two directions of discordance [30]. The conservative exact 95% interval for the overall improvement is 69.50–98.12 percentage points; for the non-Clifford primary endpoint it is 67.81–99.53 percentage points. The exact conditional test can be more conservative than asymptotic or mid-p methods [29].

Total predictions in Table 5 are obtained by adding per-condition language and native QPU predictions, including correct and incorrect conditions. Amortized time per correct task divides each condition's total runtime by four and 48, respectively, retaining the resource cost of failures. Statistical reconstruction does not require reloading the model or performing GPU inference.